

Meteorix

Loïc
Huang

Anes
Abdou

Cyprien
Renaud

Paul
Poupeau

Alex
Faucheu

Polytech Sorbonne, MAIN4
2024-2025



POLYTECH[®]
SORBONNE



SORBONNE
UNIVERSITÉ

Résumé

Le projet Meteorix vise à utiliser un nanosatellite de type CubeSat pour détecter les météores et les débris spatiaux entrant dans l'atmosphère terrestre avec une contrainte de puissance de 10W. L'objectif est de collecter des données sur ces objets grâce à une caméra embarquée, afin de mieux comprendre leur répartition et leur trajectoire. Le satellite utilise des algorithmes de détection sophistiqués pour le flot optique, et il applique des filtres pour distinguer les météores d'autres sources de lumière (éclairage, lumière des villes, etc.)

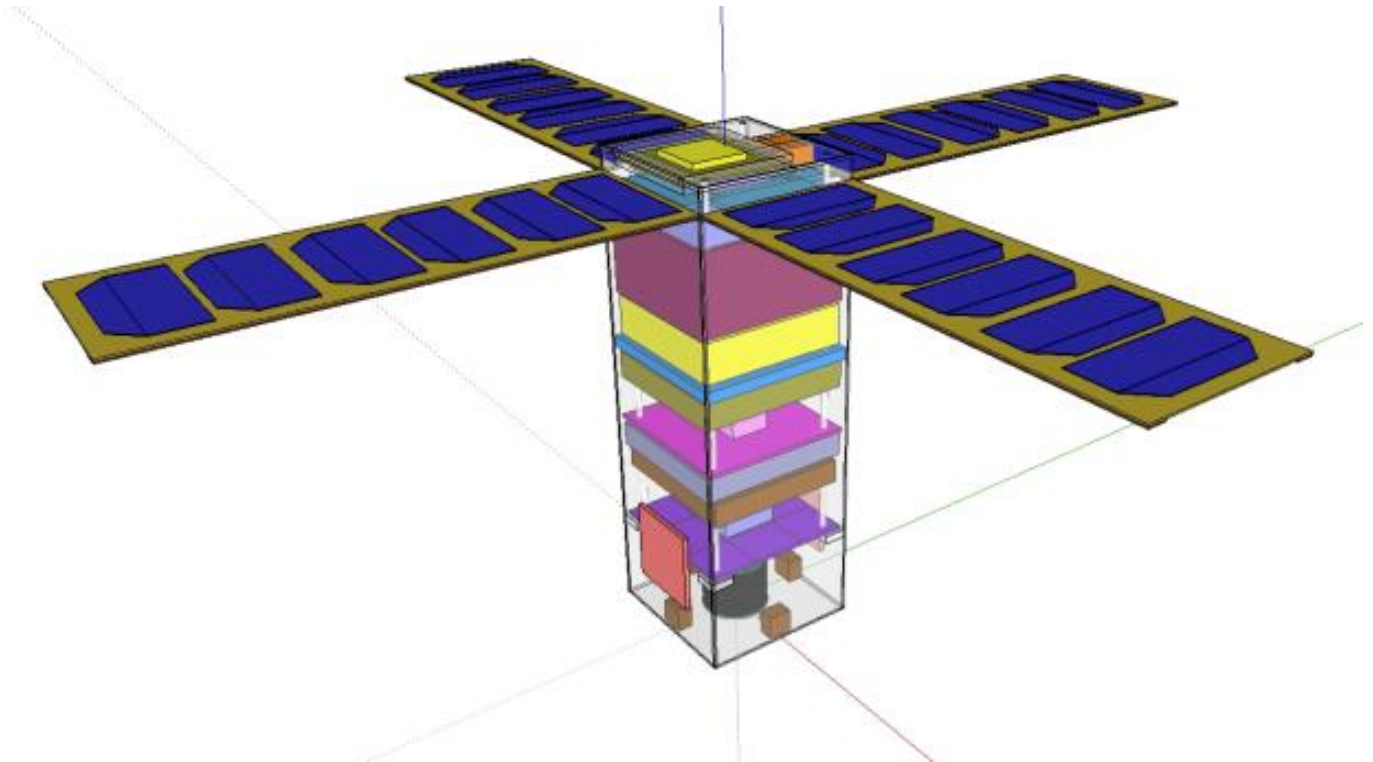


Table des Matières

1	Introduction	4
1.1	Projet global	4
1.2	Notre mission	4
2	Algorithme de détection	4
2.1	Compression vidéo	5
2.2	Filtrage des vecteurs	6
2.2.1	Motion Vector Extractor	6
2.2.2	Application de filtres	6
2.3	API C++	9
2.4	Piste d'amélioration	10
3	Environnement d'exécution	11
3.1	Procédure pour utiliser la puce matérielle	11
3.1.1	Copier les images sur la machine EM780	11
3.1.2	Transformer une vidéo compressée au format ppm	11
3.1.3	Compresser les images ppm	11
3.2	Utilisation des GPUs	12
3.2.1	NVidia GeForce RTX 3090	12
3.2.2	AMD Radeon 780M	12
4	Paramètres de compression	12
4.1	Première approche	12
4.2	Protocole expérimental	13
4.3	Commandes expérimentales	14
4.4	Analyse	14
4.4.1	Analyse visuelle	14
4.4.2	Analyse de composantes principales	15
5	Documentation pour FlowNet	18
5.1	Généralités	18
5.1.1	Définitions utiles	18
5.1.2	Flot optique	19
5.1.3	Définition CNN	20
5.1.4	Apprentissage	21
5.2	Flownet	23
5.2.1	Architecture de Flownet	23
5.2.2	Partie contractive	23
5.2.3	Partie expansive	24
5.2.4	Entraînement	25
5.2.5	Détails pratiques sur le réseau	26
5.2.6	Résultats dans l'article	27
5.2.7	Remarque sur FlowNet2	27
6	Inférence de FlowNet	28
6.1	Données testées	28
6.2	Sorties du réseau	28
6.3	Résultats	29

6.3.1	Réduction d'image	29
6.3.2	Coupage et mise en nuance de gris de l'image	30
6.3.3	Limites des tests	31
6.3.4	Conclusion	32
7	Génération d'une base de données	32
7.1	Motivations et idée générale	32
7.2	Génération d'une paire d'images	33
7.2.1	Translation du background	34
7.2.2	Mouvement du motif	34
7.2.3	Génération du flot optique associé	35
8	Conclusion	36
8.1	Impacts éthiques/sociétaux	36
8.2	Organisation	36
8.3	Pistes de recherche pour la suite	37

1 Introduction

1.1 Projet global

Meteorix est une mission CubeSat universitaire menée par plusieurs instances notamment le LIP6, le CNRS ou encore l'observatoire de Paris. La mission est dédiée à la détection et à la caractérisation des météores. L'objectif principal est d'obtenir une statistique robuste de l'entrée des météoroïdes et des débris spatiaux dans l'atmosphère terrestre.

Ces estimations permettent de quantifier l'apport de matière extraterrestre sur Terre et d'étudier les conséquences possibles en aéronomie ainsi que le risque de collisions avec les satellites artificiels lors des pluies de météores. La connaissance du flux de débris spatiaux apportera une contrainte supplémentaire pour les modèles de surveillance de l'espace développés dans les agences spatiales.

L'objectif technologique de la mission est la détection automatique des objets en temps réel à bord d'un CubeSat. Ces objectifs entrent parfaitement dans le gabarit d'une petite mission spatiale de type CubeSat 3 Unités développée par des étudiants à Sorbonne Université.

Le défi technique concerne les algorithmes de détection en temps réel des météores et, dans une moindre mesure, le contrôle de l'attitude (orientation) du satellite et sa capacité à transmettre une grande quantité de données. La détection des météores depuis l'espace est un atout essentiel pour s'affranchir des conditions météorologiques et pour couvrir une large zone du ciel.

1.2 Notre mission

Le nanosatellite envoyé dans l'espace aura une unité de calcul et une caméra. Le système de capture de météore sera limité par une contrainte énergétique de 10W pour tout le fonctionnement du système, c'est-à-dire capture de la vidéo, traitement et envoi des images.

Or un algorithme de détection efficace et satisfaisant la contrainte énergétique est déjà implémenté dans l'unité de calcul mais il fournit des images dont la qualité peut être largement améliorée en utilisant des méthodes modernes. Ce travail d'optimisation nous a été confié et est l'objectif principal de ce projet.

Pour cela nous utiliserons deux approches, la première consiste à récupérer les vecteurs issus de la compression pour essayer de détecter les zones d'intérêt d'une vidéo.

D'autre part, ces résultats pourront être utilisés pour l'autre approche qui est l'implémentation d'un réseau de neurones, FlowNet, qui permet de calculer le flot optique d'une vidéo.

De plus, une machine distante nous sera prêtée avec des propriétés similaires à l'unité de calcul qui sera envoyée dans l'espace dans le nanosatellite.

2 Algorithme de détection

L'objectif ici est de compresser les images récupérées par le nanosatellite afin d'obtenir les mouvements dans l'image et ainsi déterminer s'il y a un météore ou non.

2.1 Compression vidéo

L'idée est alors de détourner l'utilisation de la compression inter-image. En effet, la compression vidéo est une technique permettant de réduire la taille des fichiers vidéo tout en conservant une qualité d'image satisfaisante. Elle repose sur l'exploitation des redondances spatiales et temporelles afin de minimiser la quantité de données nécessaires à la reconstruction des images.

Alors, la compression inter-image exploite la similarité entre les images successives d'une vidéo. Plutôt que de stocker chaque image indépendamment, des vecteurs de mouvement sont utilisés pour décrire les déplacements des blocs d'une image à l'autre. Ainsi, seules les différences entre images sont enregistrées.

Les vidéos compressées sont généralement constituées de trois types de trames :

- I-Frames (Intra-coded Frames) : Trames autonomes qui ne dépendent d'aucune autre image
- P-Frames (Predicted Frames) : Trames prédites à partir d'une image précédente, ne stockant que les différences détectées
- B-Frames (Bidirectionally Predicted Frames) : Trames prédites en utilisant à la fois des images précédentes et suivantes, permettant une compression plus efficace

Lors de la lecture d'une vidéo, le décodeur reconstruit chaque image en combinant ces différentes trames. Ce procédé est implémenté dans divers codecs tels que H.264, H.265, MPEG-4 Part 2. Ainsi nous allons récupérer et analyser ces vecteurs pour déterminer s'il y a ou non un météore présent sur l'image. De plus, la trajectoire d'un météore est rectiligne ce qui fait qu'il est facilement identifiable si le bruit de la vidéo est faible.

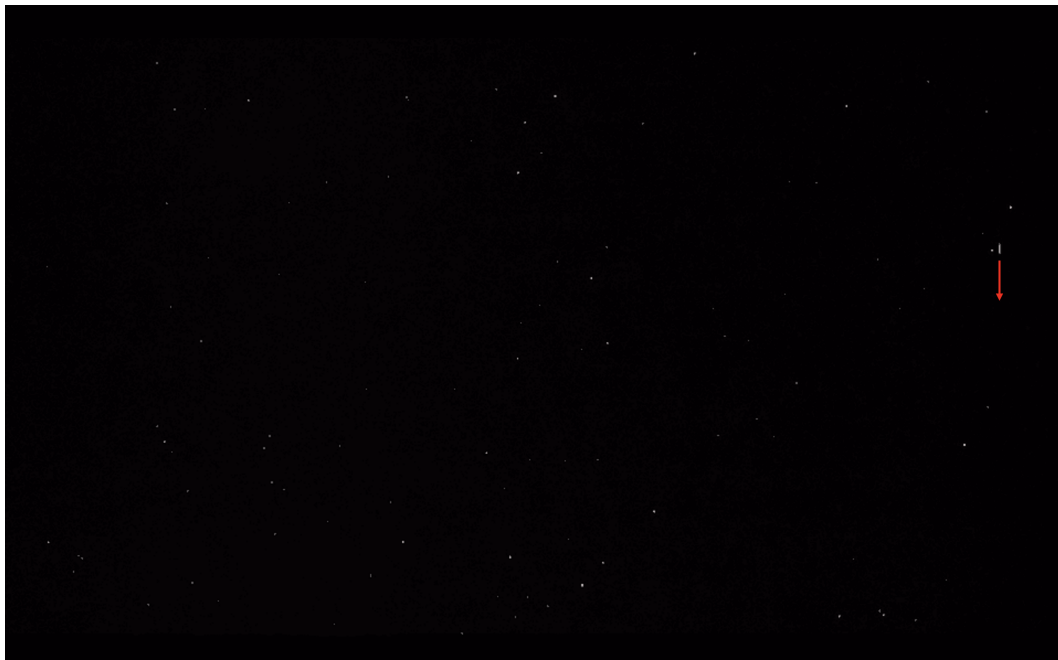


Figure 1: Représentation d'un météore

Sur l'image ci dessus, nous pouvons voir un météore sur la droite, se déplaçant vers le bas. Dans cette vidéo le météore est assez simple à détecter du fait du fond noir de la vidéo et la présence du météore sur de nombreuses images successives.

En pratique nous pourrions être confrontés à plusieurs problèmes, d'une part le météore pourrait n'apparaître que dans une seule image s'il se déplace trop rapidement (la caméra sur le nanosatellite filmera en 25 frames/s). D'autre part, le fond de la vidéo serait trop bruité, par des nuages, éclairs ou étoiles ce qui engendrerait une compression difficile à analyser.

Par exemple, en pratique sur l'image suivante, les nuages rendent la détection du météore compliquée en ajoutant du bruit.



Figure 2: Représentation d'un météore

2.2 Filtrage des vecteurs

2.2.1 Motion Vector Extractor

Pour récupérer les vecteurs nous allons utiliser l'outil Motion Vector Extractor. Cet outil extrait les images, les vecteurs de mouvement, le type des images des vidéos encodées en H.264 ou MPEG-4. Pour chaque image, on obtient donc :

- L'image décodée au format RGB
- Les vecteurs de mouvement
- Le type de l'image : I , P ou B

De plus, une API est disponible et bien documentée en Python ainsi qu'une en C++. Pour faire la compression, le logiciel FFMPEG est utilisé en arrière-plan. Par la suite nous utiliserons FFMPEG pour décoder des vidéos, compresser des images en jouant sur les paramètres de compression ou encore conserver uniquement une partie d'une vidéo en temps et espace.

2.2.2 Application de filtres

Dans cette partie nous allons montrer les résultats obtenus avec l'API Python.

Le programme initial de mv-extractor nous fournit déjà une détection de mouvements. Cependant

celui-ci ne filtre aucun vecteur, il nous est donc difficile d'identifier le météore.

Nous cherchons à trouver une solution pour conserver uniquement les vecteurs intéressants, c'est à dire ceux qui décrivent le mouvement du météore.

En première approche nous pouvons conserver les vecteurs avec une norme assez importante pour éliminer le bruit et garder l'essentiel du mouvement. Pour cela on suppose que la norme des vecteurs décrivant le météore est supérieure à celle du bruit.

L'idée suivante vient du fait que nous avons remarqué que lors de la compression, le météore est généralement décrit par plusieurs vecteurs. Ainsi nous pouvons conserver uniquement les vecteurs qui possèdent des voisins assez proches et en nombre suffisants. Pour cela on peut utiliser le module NearestNeighbors de Python qui permet de trouver les voisins les plus proches. Nous conservons le vecteur s'il a un nombre suffisant de voisins dans un cercle de rayon fixé. Ce cercle a pour origine l'extrémité finale du vecteur.

Un troisième filtre pouvant être appliqué est spécifique au météore, en effet comme dit précédemment un météore a un mouvement rectiligne. Ainsi après avoir conservé les zones avec du mouvement on filtre en choisissant celles où les vecteurs ont la même direction.

Nous avons commencé par tester nos filtres sur un extrait du court métrage Big Buck Bunny car le mouvement est assez simple à analyser.



Figure 3: Sans filtre



Figure 4: Filtre sur la norme



Figure 5: Filtre sur la norme et la zone



Figure 6: Filtre sur la norme, zone et angle

Nous pouvons remarquer qu'en combinant le filtre sur la norme et la zone cela permet de conserver les parties du lapin en mouvement telles que les bras ou les oreilles. En revanche rajouter le filtre sur les angles des vecteurs n'a pas d'utilité dans ce cas-ci. Faisons la même chose sur une vidéo de météore.



Figure 7: Sans filtre



Figure 8: Filtre sur la norme



Figure 9: Filtre sur la norme et la zone



Figure 10: Filtre sur la norme, zone et angle

Dès l'application du filtre sur la norme et la zone nous arrivons à isoler le météore. En revanche dans la même vidéo il y a un éclair qui, même en appliquant les 3 filtres, reste visible. Nous pourrions appliquer des filtres plus forts mais nous risquons de ne plus détecter le météore.



Figure 11: Image de météore

Par la suite, un algorithme de type knapsack pourra être implémenté. L'idée serait de maximiser une zone en faisant la somme des normes des vecteurs vitesse tout en minimisant le nombre de vecteurs utilisés et ainsi obtenir la zone de plus forte densité.

2.3 API C++

Notre projet ayant pour objectif d'utiliser le moins d'énergie possible, nous avons choisi de traduire le code Python en C++, ce dernier étant un langage de plus bas niveau et plus performant.

Tout comme le programme Python, nous avons amélioré l'API C++ en implémentant des fonctions permettant de filtrer les vecteurs selon leur norme, leur nombre de voisins, et d'éliminer tous les vecteurs isolés.

Pour réaliser cela, il est nécessaire d'installer préalablement les bibliothèques OpenCV et FFMPEG.

```
sudo apt-get update
sudo apt-get install libavcodec-dev libavformat-dev libavutil-dev
libswscale-dev libopencv-dev
```

Nous avons aussi ajouté un Makefile sur mv-extractor pour que la compilation soit automatique.

Pour lancer le programme il suffit d'exécuter la commande :

```
./main -v input.mp4 -m -r 20.0 -n 10 -p 1 -V 1 -d output/
```

Nous avons ajouté une fonction `generate_timestamp_folder()` qui utilise les fonctionnalités de la bibliothèque `<ctime>` afin de générer une chaîne au format "YYYYMMDD_HHMMSS".

Pour la commande d'exécution voici la liste des options disponibles :

- `-m` : Seuil de magnitude.
- `-r` : Rayon de proximité.
- `-n` : Nombre minimum de voisins requis.

- `-p` : Prévisualisation.
- `-V` : Mode verbeux.
- `-h` : Afficher l'aide.

Enfin, nous avons comparé les temps d'exécution des versions Python et C++ en variant la norme minimale des vecteurs à conserver. C'est-à-dire qu'il y aura de plus en plus de vecteurs à supprimer mais moins à imprimer sur l'image.

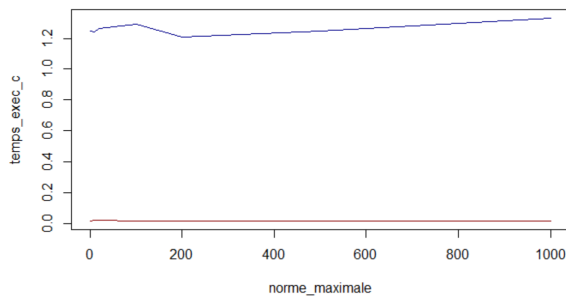


Figure 12: Comparaison du temps d'exécution du programme Python vs C++

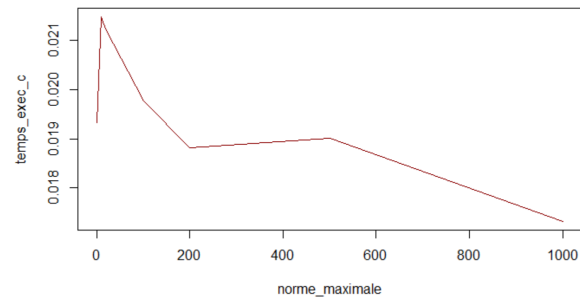


Figure 13: Temps d'exécution c++

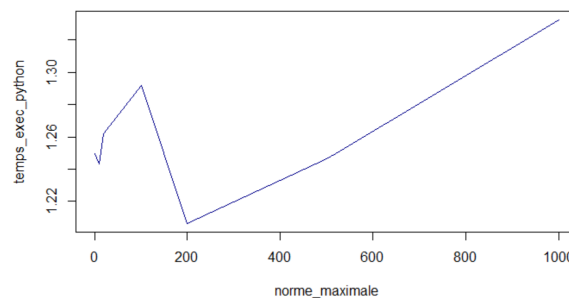


Figure 14: temps d'exécution python

Comme attendu, la version C++ est nettement plus performante que celle en Python. Elle est presque 100 fois plus rapide que la version Python.

2.4 Piste d'amélioration

Pour optimiser davantage l'algorithme de détection de météores et de filtrer plus efficacement les mouvements qui ont lieu sur les images, on pourrait envisager le ciblage sur le GPU embarqué qui pourrait accélérer significativement les calculs, notamment pour le traitement des nombreuses vidéos, demandant beaucoup d'énergie. On pourrait également implémenter une fonction pour filtrer plusieurs images successives qui sont obtenues à partir d'une séquence vidéos, ce qui permettrait d'affiner la détection en ayant une précision bien meilleure sur la trajectoire des météores et leurs caractéristiques. Enfin, on pourrait aussi augmenter le nombre de vidéos testées avec des paramètres variés, cela permettrait en particulier d'enrichir la base de données pour l'analyse fonctionnelle PCA, améliorant le modèle. Ces pistes donneraient alors une automatisation plus efficace et précise de la détection de météores.

3 Environnement d'exécution

Dans l'espace, le nanosatellite sera équipé d'une puce matérielle pour encoder une vidéo à partir d'images brutes. Notre client, Mr. Cassagne, nous a mis à disposition l'EM780, une machine distante située au Laboratoire d'Informatique de Paris 6.

3.1 Procédure pour utiliser la puce matérielle

Pour utiliser la puce matérielle nous avons suivi la procédure suivante :

3.1.1 Copier les images sur la machine EM780

Nous avons des vidéos de météores en local, nous allons copier les vidéos sur le disque dur de l'EM780 qui a des encodeurs similaires à ceux que l'on aura sur le satellite.

Pour se connecter à la machine :

```
ssh em780.mono.lip6.ext
```

Puis on copie les vidéos de la manière suivante :

- (en local) `scp fichier-sur-la-machine em780.mono.lip6.ext:~/`
- (sur l'em780) `mv v03.mp4 /scratch/<username>-nfs`

De plus, FFMPEG est déjà installé sur la machine, on pourra donc utiliser directement les commandes FFMPEG depuis la machine.

3.1.2 Transformer une vidéo compressée au format ppm

Pour le moment nous avons des vidéos de météores dans des formats compressés (H.264 ou MPEG-4) que nous avons pu récupérer sur l'EM780. Or la caméra du satellite récupère des images au format ppm (format brut). Nous voulons donc transformer nos vidéos compressées en images au format ppm pour ensuite donner ces images à la puce afin qu'elle les compresse. La commande suivante permet de le faire en créant un répertoire stockant les images :

```
mkdir -p output_images && ffmpeg -i video.mpg output_images/image%d  
.ppm
```

3.1.3 Compresser les images ppm

Enfin pour compresser les images nous allons adresser l'un des deux GPUs présent sur l'EM780

- adresser la puce sur le GPU embarqué AMD Radeon 780M (avec VA-API)
- adresser la puce sur le GPU externe NVidia GeForce RTX 3090 (dans ce cas il faut utiliser NVENC)

L'option 1 est la meilleure car la puce est sur le SoC, c'est-à-dire que le GPU est directement sur la puce ce qui est plus proche de la réalité dans un nano-satellite. L'option 2 est certainement plus facile car NVENC est bien supporté et documenté en général.

3.2 Utilisation des GPUs

3.2.1 NVidia GeForce RTX 3090

Nous avons réussi à utiliser le GPU Nvidia avec la commande suivante :

```
ffmpeg -hwaccel cuda -framerate 30 -i output_images/image%d.ppm -c:v h264_nvenc -preset fast -b:v 5M compressed_video.mp4 -loglevel verbose
```

Ce qui est important à retenir est `-hwaccel cuda` qui active l'accélération matérielle CUDA pour optimiser le traitement ainsi que `-c:v h264_nvenc` qui utilise NVENC pour encoder la vidéo au format H.264. Dans la commande on donne le répertoire avec les images au format ppm et on indique le nom et le format de la vidéo en sortie.

On peut aussi regarder l'utilisation du GPU en parallèle avec `nvidia-smi` et comparer la consommation d'énergie avant et pendant la compression. Nous trouvons une consommation de 28W sans processus en cours tandis que la consommation est de 108W en lançant notre algorithme. Il y a donc une augmentation de 80 watts, nous pouvons donc conclure que FFMPEG utilise bien le GPU. Cela nous permettra de simuler la compression dans l'espace.

3.2.2 AMD Radeon 780M

Pour utiliser le GPU AMD Radeon on va essayer d'utiliser la bibliothèque VAAPI. Avec la commande suivante on adresse l'AMD Radeon 780M :

```
ffmpeg -vaapi_device /dev/dri/renderD128 -i output_images/image%d.ppm -vf 'format=nv12,hwupload' -c:v h264_vaapi -qp 24 -preset fast compressed_video.mp4
```

L'idée est la même que précédemment avec `-vaapi_device /dev/dri/renderD128` qui spécifie le périphérique VAAPI à utiliser, `-vf 'format=nv12,hwupload'` qui convertit les images au format compatible VAAPI (nv12) et les télécharge sur le GPU. Et enfin `-c:v h264_vaapi` qui utilise VAAPI pour encoder en H.264.

Nous n'avons pas réussi à utiliser l'AMD Radeon du fait de bibliothèques trop compliquées à installer. Pour la suite nous utiliserons donc uniquement le GPU NVidia GeForce RTX 3090.

4 Paramètres de compression

Dans cette partie nous allons étudier les paramètres de compression pour déterminer la manière la plus efficace de capturer un météore si on applique notre algorithme de filtrage sur une vidéo compressée par l'EM780.

4.1 Première approche

Tout d'abord pour lancer Motion Vector Extractor sur l'EM780 nous avons eu l'idée de créer un environnement virtuel Python et ainsi installer les bibliothèques nécessaires au script.

Comme première approche nous allons lancer notre algorithme sur une vidéo compressée avec les paramètres par défaut sur l'EM780 et comparer les vecteurs que l'on obtient avec celle que notre machine personnelle a extraite.



Figure 15: Compression originale



Figure 16: Compression avec l'EM780

Nous remarquons que notre algorithme a réussi à capturer le météore sur notre PC au contraire de l'EM780. Les filtres appliqués sont peut être trop forts, ou l'image est simplement compressée différemment.

4.2 Protocole expérimental

Nous avons vu qu'en encodant la vidéo avec le GPU sur l'EM780 les vecteurs vitesse trouvés n'étaient pas bons. Nous allons donc essayer de jouer sur les paramètres d'encodage pour obtenir un résultat satisfaisant.

Nous pouvons modifier :

- Le mode d'encodage, on peut encoder en H.264 ou H.265
- Le framerate, soit le nombre d'image par seconde (par défaut, 25)
- Le preset, plus le preset est faible, plus la compression prendra du temps mais la qualité de l'image sera moins dégradée
- La fréquence des images clés (par défaut, framerate*10)
- Le bitrate, on peut choisir un bitrate fixe ou alors variable qui adapte la compression en donnant plus de bits en partie en mouvements

Nous allons suivre le déroulé suivant :

1. On commence par compresser les images avec les commandes décrites dans 4.3
2. On lance Motion Vector Extractor sur chaque vidéo
3. On transfère les données obtenues sur notre PC afin de les analyser

4.3 Commandes expérimentales

Numéro	Framerate	Preset	Images clés	Bitrate	Encodage
1	30	fast	300	fixe	H.264
2	30	fast	10	fixe	H.264
3	30	slow	10	fixe	H.264
4	30	slow	5	fixe	H.264
5	25	slow	10	fixe	H.264
6	25	p7	10	fixe	H.264
7	30	p7	10	variable	H.264
8	30	p7	10	variable	H.264
9	30	slow	15	fixe	H.264
10	30	slow	10	fixe	H.265
11	30	p7	10	fixe	H.264

Table 1: Commandes de compression

4.4 Analyse

Nous allons regarder quelles sont les combinaisons de paramètres qui capturent au mieux un météore.

4.4.1 Analyse visuelle

Les images suivantes sont celles où le météore a été le mieux attrapé en utilisant les paramètres du tableau ci-dessus. Sur les images la tache blanche est le météore et les vecteurs sont issus de la compression. La première remarque que l'on peut faire est que l'encodage H.265 n'est pas supporté (Combinaison 10). Ensuite il ne faut pas laisser un nombre trop faible d'images clés sous peine de ne même pas capturer le météore (Combinaison 1).

En revanche il est difficile de déterminer quelle est la meilleure combinaison, c'est pourquoi nous allons faire une analyse de composantes principales pour essayer d'y répondre.



Figure 17: Combinaison 1

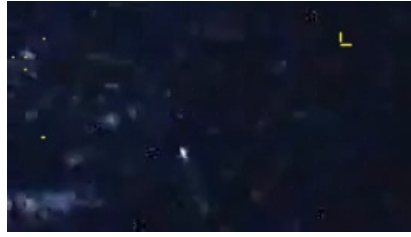


Figure 18: Combinaison 2



Figure 19: Combinaison 3

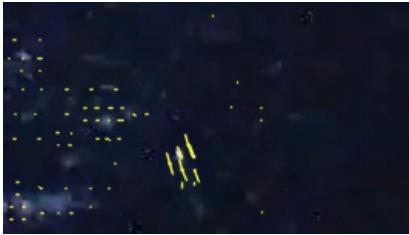


Figure 20: Combinaison 4

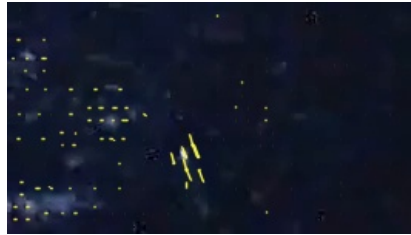


Figure 21: Combinaison 5

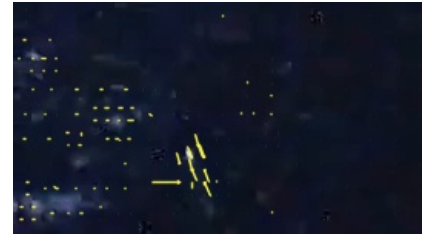


Figure 22: Combinaison 6



Figure 23: Combinaison 7



Figure 24: Combinaison 8



Figure 25: Combinaison 9

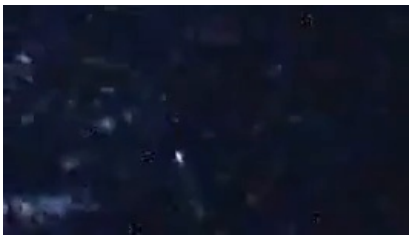


Figure 26: Combinaison 10



Figure 27: Combinaison 11

4.4.2 Analyse de composantes principales

L'analyse en composantes principales (ACP) est une méthode statistique permettant de réduire la dimensionnalité d'un jeu de données tout en conservant l'essentiel de l'information. Elle transforme un ensemble de variables corrélées en un nouveau système de variables non corrélées, appelées composantes principales, ordonnées selon la variance expliquée.

Dans le cadre de l'optimisation des paramètres de compression pour la détection de météores en vidéo, l'ACP va nous permettre d'identifier les facteurs influençant le mieux la qualité de détection. Ainsi, l'ACP constitue un outil pertinent pour sélectionner les configurations de compression opti-

males.

Dans un premier temps, nous avons collecté les données de chaque frame tel que la moyenne des composantes x et y des vecteurs, la norme ainsi que l'écart type des composantes x et y. Nous associons à chaque frame les paramètres de compression utilisés ainsi qu'un facteur indiquant si le météore a été capturé (1) ou non (0).

Pour cela on utilise l'analyse visuelle pour savoir s'il y a suffisamment de vecteurs sur le météore pour dire s'il a bien été attrapé.

Enfin, nous lançons l'ACP en utilisant le package FactoMiner de R qui permet de centrer et réduire les données afin qu'elles aient toutes la même échelle.

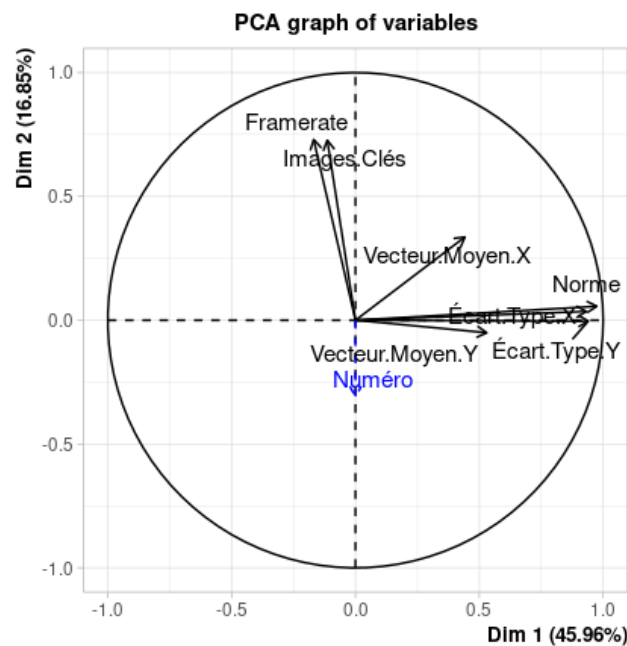


Figure 28: Cercle des corrélations

La première remarque que l'on peut faire est que les deux premières composantes principales expliquent 60 % de l'inertie du jeu de données, ce qui signifie que ces deux axes capturent 60 % de la variance totale des données. Cela permet de réduire la dimensionnalité du jeu de données tout en préservant une part significative de l'information.

Ensuite, le premier axe oppose les images avec un vecteur de norme et d'écart type élevé aux images de norme faible ou négative et d'écart type nul. Le second axe, quant à lui, oppose les images d'une vidéo compressée avec peu d'images clés et un framerate élevé avec son inverse.

Maintenant passons à la représentation des individus dans le premier plan principal. Nous allons uniquement présenter les résultats intéressants.

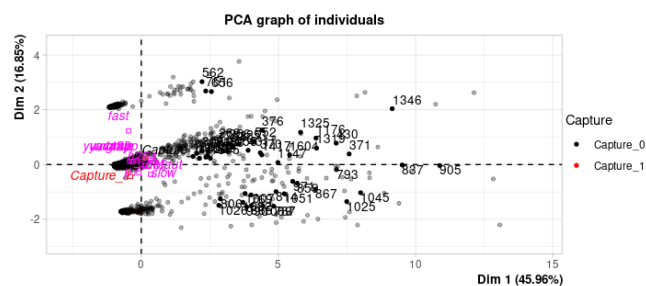
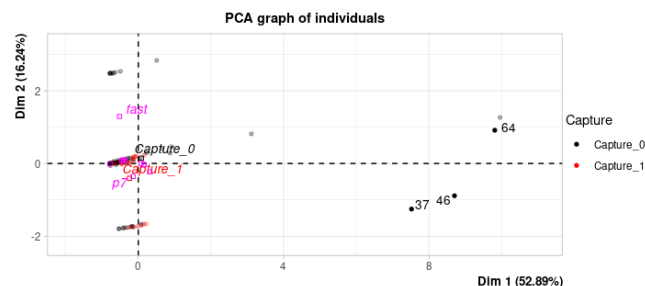


Figure 29: Toutes les frames



5 Documentation pour FlowNet

Avant de commencer à implémenter et tester le réseau, nous avons commencer par nous documenter sur le réseau et comment fonctionne t-il. Nos sources sont citées dans la bibliographie.

5.1 Généralités

Le but de cette section est d'introduire des notions générales afin de mieux comprendre FlowNet.

5.1.1 Définitions utiles

Dans cette section nous définissons des termes retrouvées très souvent dans d'autres parties. Il serait peut-être intéressant de lire une première fois ces définitions puis d'y revenir souvent en lisant le reste de la documentation.

Canal (ou "feature map"): Classiquement, le **canal** est une des trois images élémentaires qui composent une image couleur (voir illustration ci-dessous). Néanmoins, en traitement d'images (et notamment dans notre cas) il est possible d'étendre la définition de canal. Un canal est alors une couche de l'image apportant des informations sur celle-ci (donc pas nécessairement la couleur). Par exemple, un canal peut représenter les contours de l'image, un autre peut représenter les zones lumineuses de l'image... En général, un canal représente une caractéristique de l'image. Dans un CNN, en pratique, un canal est une matrice. Les canaux peuvent aussi être appelés **patch**.



Figure 33: Canaux d'une image

Une **coarse feature** est un canal à l'intérieur du réseau qui contient les informations de l'image de manière "grossière" (coarse en anglais) avant le produit rendu par le réseau.

Le **pooling** est une étape dans les réseaux de neurone type CNN où la taille des features sont réduites petit à petit pour extraire les données voulues.

L'**unpooling** après le pooling consiste à augmenter la taille des canaux pour retrouver une image avec une bonne résolution.

L'**upconvolution** est une sorte d'unpooling qui consiste à un upsampling (augmentation de la taille de la feature) puis une convolution.

Le **bed of nails** est une méthode d'upsampling qui augmente la taille d'un canal en gardant la donnée en haut à gauche et place des zéros autour (en bas et à droite).

Les **frame pairs** est la paire d'images où on cherche à établir le flux optique.

Les données "**ground truth**" représente dans notre cas le flux optique réel.

Le "**ground truth density per frame**", en français le pourcentage de données réel par image permet de donner le pourcentage de l'image où on a le flux optique réel (si c'est à 50% la moitié de l'image n'a pas de données réelles).

La méthode de **stride of 2** est une couche de convolution où le masque au lieu de se déplacer de "+1" position le masque se déplace de "+2" positions, ce qui divise la taille de moitié de la feature map.

Le **ReLU** est une fonction d'activation qui met à 0 des données dépassant un certain seuil (dans notre cas, qui sont négatives).

L' "**End-to-end Point Error**" est l'erreur standard pour l'estimation du flux optique où le flux optique estimé (pour un pixel donné : v_{est}) est comparé avec les données GT (ground truth) (pour un pixel donné : v_{gt}) et est calculée comme la distance euclidienne entre les deux vecteurs :

$$||v_{est} - v_{gt}||$$

Pour calculer l'EPE d'une image entière, une moyenne est calculée sur tout les EPEs.

5.1.2 Flot optique

À partir des images avec la zone d'intérêt extraites grâce à MVE, nous allons maintenant chercher à calculer le flot optique.

On considère l'application I qui quantifie l'intensité lumineuse d'un pixel dans une des images (en faisant la simplification que l'image est en nuances de gris par exemple). L'intensité lumineuse d'un pixel de coordonnées (x, y) à l'instant t est donc donnée par $I(x, y, t)$.

Dans la suite de cette partie on cherche à déterminer le flux optique entre 2 images successives séparées d'un délai de dt .

Premièrement, il est nécessaire de faire l'hypothèse d'illumination constante qui consiste à dire que $I(x + dx, y + dy, t + dt) = I(x, y, t)$

Par ailleurs on peut appliquer la formule de Taylor à l'ordre 1 sur I pour dx, dy, dt proches de 0 :

$$I(x + dx, y + dy, t + dt) = I(x, y, t) + \frac{\partial I}{\partial x}(x, y, t) dx + \frac{\partial I}{\partial y}(x, y, t) dy + \frac{\partial I}{\partial t}(x, y, t) dt$$

Or, d'après l'hypothèse qu'on a faite, on obtient :

$$\frac{\partial I}{\partial x}(x, y, t) dx + \frac{\partial I}{\partial y}(x, y, t) dy + \frac{\partial I}{\partial t}(x, y, t) dt = 0$$

D'où en "divisant" par dt (non nul) on a :

$$\frac{\partial I}{\partial x}(x, y, t) \frac{dx}{dt} + \frac{\partial I}{\partial y}(x, y, t) \frac{dy}{dt} + \frac{\partial I}{\partial t}(x, y, t) = 0$$

Depuis cette équation on peut définir le vecteur de flux optique au point (x, y) à l'instant t qui correspond au vecteur $u = (\frac{dx}{dt}, \frac{dy}{dt})$. Ce dernier peut-être vu comme le vecteur vitesse du point image (x, y) à l'instant t .

Conclusion : On peut interpréter le flux optique entre 2 images successives comme étant l'ensemble des vecteurs vitesses à l'instant t associés à chaque point image. Plus généralement, dans une vidéo, on peut considérer que le flux optique est la vitesse de chaque point image (fonction à 2 composantes) en faisant l'hypothèse de l'illumination constante.

5.1.3 Définition CNN

Un CNN est un cas particulier de réseau de neurones. En effet, il présente comme les autres réseaux une couche d'entrée (qui correspondra à l'image d'entrée) et une couche de sortie (qui correspondra en général à une couche de classification)¹. Les CNN présentent 3 types de couche : les couches de convolution, les couches de pooling et les couches entièrement connectée (FC)²:

- Les couches de convolution ont une architecture de sorte que la propagation du signal au travers de cette couche effectue un produit de convolution avec celui-ci. La sortie de cette couche correspondra au produit de convolution entre la **sortie de la couche précédente** et un **masque de convolution** correspondant aux poids neuronaux (associés aux connexions neuronales entre la couche précédente et la couche de convolution).
- Les couches de Pooling servent essentiellement à réduire la taille de l'image entre 2 couches de convolution. Cela permet de réduire la complexité du réseau et de le rendre ainsi utilisable en pratique. On parle souvent de max-pooling car on utilise en général une fonction qui prend le maximum de l'intensité du voisinage d'un neurone/pixel.
- Les couches FC sont en général à la fin du réseau pour "interpréter" les "**features maps**" ou **cartes caractéristiques** de l'image obtenues après la succession de couches de convolutions et de pooling. A la suite de ces couches on obtient une sortie qui en général (ce n'est pas le cas de FlowNet) classe quelque chose dans l'image de départ (comme des chats par exemple).

Finalement, on notera que la force des CNN pour le traitement d'image (par rapport aux réseaux classiques) semble être cette astuce de limiter le nombre de paramètres entre chaque couche de convolutions. En effet, si on souhaite faire du traitement d'image avec un réseau naïf de neurones (avec

¹FlowNet, lui, n'est pas un classifieur, sa sortie correspond bien à une image c'est une différence significative par rapport aux CNN classiques.

²FlowNet semble remplacer ces couches par l'étape de "refinement" (voir la figure 1 plus bas) pour obtenir une image en sortie plutôt qu'une classification.

que des couches FC), le nombre d'entrées étant très grand pour une image, le nombre de paramètres entre chaque couche exploserait. Tandis qu'avec un CNN, si on choisit des noyaux de convolution 3x3, on se limite à 9 paramètres pour chaque couche de convolution ce qui réduit drastiquement la complexité du réseau.

5.1.4 Apprentissage

Descente de gradient :

Dans le cas de FlowNet, l'apprentissage par descente de gradient se fait (sur une couche) sur une matrice w des poids en fonction de "vecteurs" de pixels ³.

Avec α un réel le learning rate et pour l'itération n , on a la formule :

$$w_n = w_{n-1} - \alpha \nabla J(w_{n-1})$$

J étant une fonction de mesure de l'erreur avec comme données d'entraînement $\{(x^{(i)}, y^{(i)})\}_{i \in \{1, \dots, m\}}$, m le nombre de paire de données, où $x^{(i)}$ la donnée d'entrée et $y^{(i)}$ le flux optique réel ("GT") :

$$J(w_{n-1}) = \frac{1}{2m} \times \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$$

h étant la fonction qui associe à $x^{(i)}$ le flux optique prédit par le réseau :

$$h(x^{(i)}) = w_0 + \sum_{j=1}^n w_j x_j^{(i)}$$

Ainsi, avec $x^{*(i)} = [1, x^{(i)}]$ pour prendre en compte w_0 :

$$\nabla J(w_{n-1}) = \frac{1}{2m} \times \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) \times x^{*(i)}$$

La descente de gradient peut être utilisée de 3 manières différentes :

- batch : on utilise **toutes** les données de la data d'entraînement dans une matrice, et on calcule le gradient sur toute les données mais cela ralentit l'apprentissage.
- stochastique : on sélectionne une donnée aléatoire à la i -ème itération en pensant que cette donnée fiable ressemble à toute les autres⁴
- mini-batch : on sélectionne de manière aléatoire une partie des données d'entraînement ⁵

Probleme de l'underfitting et de l'overfitting :

A partir de l'apprentissage par descente de gradients stochastique ou en mini-batch, le problème

³“Since, in a sense, every pixel is a training sample” dans l'article

⁴Dans notre cas, ce n'est pas possible, ça voudrait dire utiliser qu'une seule "position" et pour le calcul de flux optique nous avons besoin de plusieurs pixels côte à côte

⁵Pour FlowNet, ils ont pris 8 paires d'images comme taille pour une itération

de l'underfitting et l'overfitting est liée au nombre d'itérations effectué pour entraîner le réseau. En effet, l'underfitting est le fait de ne pas assez entraîner le réseau alors que l'overfitting entraîne "trop" le réseau et alors perd sa capacité à généraliser.

Pour faire face à ce problème, il existe différentes méthodes. Dans le cas de FlowNet, ils ont utilisé un "**validation set**" qui teste à chaque itération quels sont les meilleurs poids à garder.

Finalement, le **fine-tuning** permet de "spécialiser" un réseau de neurones sur un jeu de données précis après qu'il ait été complètement entraîné avec le nombre d'itérations optimal trouvé lors de l'entraînement.

Backpropagation :

Le principe de backpropagation permet d'entraîner un réseau de neurone avec plusieurs couches. Cela consiste à calculer la sortie avec les poids existants au temps t puis changer les poids en partant de la fin et en se basant sur le principe (avec $z(x, y)$, $x(s)$ et $y(s)$)⁶ :

$$\frac{\delta z}{\delta s} = \frac{\delta z}{\delta x} \times \frac{\delta x}{\delta s} + \frac{\delta z}{\delta y} \times \frac{\delta y}{\delta s}$$

Algorithme d'Adam :

L'algorithme d'ADAM est un algorithme qui modifie le learning rate pour optimiser l'apprentissage dans la méthode de descente de gradient. Pour essayer de comprendre l'algorithme, nous allons expliciter les optimisations faites de manière temporelle :

- **AdaGrad** : La première optimisation a été de diminuer le learning rate en fonction du temps et du gradient (et ainsi de prendre en compte le fait que l'algorithme converge vers la solution). Nous avons donc, avec δ une valeur pour ne pas diviser par 0, ϵ le learning rate de départ :

$$r \leftarrow r + \nabla J(w_{t-1})^2$$

$$w_t \leftarrow w_{t-1} - \frac{\epsilon}{\delta + \sqrt{r}} * \nabla J(w_{t-1})$$

Cet algorithme a cependant un défaut : le learning rate ne fait que diminuer, de manière assez brutale, avec le temps.

- **RMSProp** : Pour ralentir la progression, une restriction a été mise sur r . Une moyenne logarithmique est faite entre le terme précédent et le terme rajouté :

$$r_t \leftarrow \gamma * r_{t-1} + (1 - \gamma) * \nabla J(w_{t-1})^2$$

- **ADAM** : La dernière optimisation est de prendre en compte le **moment** (s), qui est une variable qui permet de ne pas tomber dans un minimum local et de chercher le minimum

⁶avec FlowNet nous avons comme fonction d'activation ReLU

global. Mathématiquement, c'est une variable qui change en fonction du gradient au premier ordre (plus le gradient est grand, plus le moment est grand et plus le learning rate est élevé). ⁷

5.2 Flownet

Cette section se concentre sur le fonctionnement de FlowNet.

5.2.1 Architecture de Flownet

FlowNet est un réseau de neurone qui n'a pas eu besoin d'un dataset réaliste pour avoir des résultats intéressants. Le réseau est capable de généraliser et il est même meilleur que certaines méthodes à la pointe (à ce moment-là) tel que Deepflow et Epicflow sur ses données d'entraînement. Le code source est disponible dans la bibliographie.

Pour calculer le flot optique, la taille des entrées et sorties étant très grande, les couches de pooling sont nécessaires pour la praticité du réseau de neurones. De plus, nous en avons besoin pour pouvoir agréger des parties de l'image en flux "globaux" optique. Faire du pooling réduit la résolution de l'image, nous avons donc une partie du réseau qui "pool", **la partie contractive**. Puis, la deuxième partie du réseau réaugmente la taille de l'image (pour pouvoir fournir le flot optique demandé), **la partie expansive** dite de "**refinement**". Ce réseau est entraîné en tant qu'une seule entité (pas d'entraînement différents entre les deux parties), par rétropropagation du gradient (nous en parlerons dans une autre section).

5.2.2 Partie contractive

- FlownetSimple : les **deux images sont données dans la même entrée** au réseau et il cherche "seul" la correspondance entre les deux images et le flux optique. Plus précisément, on "superpose" les deux images dans 6 canaux.
- FlownetCorr : Les deux images ont un prétraitement où leurs caractéristiques sont extraites en coarse features dans deux "branches" indépendantes du réseau en parallèle puis sont regroupées.

Détails sur la corrélation :

Les deux feature maps sont associées par partie (appelée patch) par une somme de produit scalaire appelée **multiplicative patch comparison**.

Pour comprendre comment ça fonctionne, regardons la formule d'une simple comparaison de deux patches centrés sur la position x_1 dans la première map et x_2 dans la deuxième :

$$c : (w \times h) \times (w \times h) \rightarrow \mathbb{R}$$

$$c(x_1, x_2) = \sum_{o \in [-k, k] \times [-k, k]} (\langle f_1(x_1 + o), f_2(x_2 + o) \rangle)$$

Ici les deux patches sont de taille $(2k + 1) \times (2k + 1)$ ⁸ et f_1 (respectivement f_2) représentent les données d'un pixel sur toutes les coarse features d'entrée de l'image 1 (respectivement de l'image 2), autrement dit, sur toute la profondeur de l'image en position $x_1 + o$ ou $x_2 + o$. Ici, p représente le

⁷Le learning rate diminuant moins, dans le cas de FlowNet, on voit que l'algorithme est lancé plusieurs fois avec des learning rate différents

⁸Ici $k=0$

nombre de feature maps :

$$f_1, f_2 : w \times h \rightarrow \mathbb{R}^p$$

Si on cherche les correspondances pour tous les points de la première feature map avec la deuxième, le réseau ne sera pas entraînable. En effet, nous aurons trop de données à calculer. Aussi, deux pixels très loin l'un de l'autre avec la même intensité lumineuse pourraient correspondre alors que ce n'est pas ce que l'on veut. Nous pouvons avoir une image avec deux étoiles par exemple où le réseau pourrait déduire un flot optique entre les deux.

Pour pallier à ce problème, une limitation sur le décalage entre x_1 et x_2 a été mise en place (recherche de correspondance entre x_1 et x_2 seulement dans leur voisinage respectif). Le déplacement entre les deux feature maps a été mis en place⁹ en fonction de la limitation. Le voisinage autorisé est une fenêtre de diamètre $D = 2d + 1$ ¹⁰.

La corrélation est de dimension 4 et pour chaque combinaison de position on a une valeur de corrélation. En pratique, le déplacement relatif est organisé dans D^2 canaux de taille $w \times h$. Pour l'apprentissage, les derivatives sont implémentées en connaissant les données d'entrées¹¹.

5.2.3 Partie expansive

Upconvolutionnal layer : le principe est d'étendre (en largeur et hauteur) les feature maps à la fin de la partie contractive. Pour cela on fait un **bed of nails** et une convolution avec un masque de notre entrée (la fin de notre partie contractive), puis une concaténation avec des feature map de la partie contractive **gardées en mémoire**.¹²

Cette couche a été grandement inspiré d'un autre travail scientifique :

"we perform unpooling by simple "bed of nails" upsampling, that is, replacing each entry of a feature map by an $s \times s$ block with the entry value in the top left corner and zeros elsewhere. This increases the width and the height of the feature map s times. We used $s = 2$ in our networks. When a convolutional layer is preceded by such an upsampling operation we can think of upsampling+convolution ("up-convolution")"

Les données sauvegardées des précédentes convolutions sont utilisées comme informations à ajouter sur les coarse feature maps de la fin de la première partie.

Cette méthode permet de garder les informations des coarse feature maps (le "flot optique global") et des informations précises locales avec les niveaux précédents.

Ils se sont arrêtés à une résolution de l'image (de sortie) de 96x128 pour un input de 384 x 512 (la taille est divisée par 4). Ce format donne déjà les informations nécessaires sur le flot optique et plus de couches dans la partie expansive n'améliorent pas le résultat. Il réalise une interpolation bilinéaire avec la sortie pour avoir la bonne taille de l'image d'entrée.

⁹Ici $s_1=1$ et $s_2=2$

¹⁰Ici $d=20$

¹¹à revoir si on utilise un jour otowNetCorr

¹²L'article parle aussi d'utiliser un flot de prédiction grossier si disponible sans doute pour l'apprentissage

variante : (+v)

Le “**coarse to fine scheme**” est un algorithme qui permettrait de diminuer l’erreur d’un flux optique généré, par exemple, par un CNN et l’affiner.

Ils expliquent qu’ils ont pris en compte le léger changement de couleur et de luminosité pour diminuer l’erreur.

A partir de la sortie donnée du réseau de neurone, il ont utilisé le “coarse to fine scheme” pour augmenter la taille de l’image jusqu’à la taille initiale (sur 20 itérations) et affiner le flux (sur 5 itérations).¹³

5.2.4 Entraînement

FlowNet étant novateur (à ce moment-là) et le flot optique étant compliqué à calculer avec des images réelles, trouver des données d’entraînement adaptées est difficile.

Détails sur les jeux de données existants :

- Middlebury : très petit nombre de données et de très petits mouvements
- KITTI : mouvements spécifiques avec un pourcentage faible (environ 50 pour cent) de données vérifiées sur le flot optique “GT” (les données sont réelles : elles ne sont pas générées artificiellement)
- MPISintel : nombre de données considérable, GT avec un pourcentage de 100 pour cent (de certitude) mais pas assez pour entraîner un réseau de neurone (base de données de synthèse)

Pour pouvoir entraîner le réseau, ils ont créé un jeu de données, le Flying Chair Dataset :

L’article donne des détails sur sa création.

Ils ont choisi aléatoirement :

- des transformations affines en 2D sur le fond et sur les chaises pour les déplacements
- La taille, l’endroit où les chaises se placent

La distribution utilisée pour générer les données aléatoires est ajustée pour que les données soient similaires à celle de MPISintel¹⁴. Les créateurs de Flying Chairs auraient pu apparemment générer plus de données pour le dataset mais se sont arrêtés à 22 872 paires d’image.

Pour éviter le problème de l’**overfitting** (définie section apprentissage : 5.1.4), la data augmentation a été utilisée sur :

- la translation
- la rotation
- la taille
- le bruit gaussien
- la luminosité
- le gamma

¹³plus de détails dans l’article, ce n’est pas intéressant dans notre cas

¹⁴de la documentation supplémentaire est disponible sur le sujet, pour l’entraînement du réseau qu’on va implémenter ça peut être intéressant

- le contraste
- la couleur

Les transformations ont été faites sur la paire d'images entière **et** entre les deux images de chaque paire (avec des modifications plus petites, les détails sont dans l'article).

5.2.5 Détails pratiques sur le réseau

FlowNet est composée de 9 couches de convolution. Parmi ces couches de convolution, 6 d'entre elles effectuent aussi du pooling avec la méthode du **stride of 2** où la taille des feature maps est divisée par 2 et le nombre de feature maps est multiplié par 2. Un ReLU est présent après chaque couche de convolution.

De plus, on peut avoir en entrée du réseau une taille variable entre les paires d'images car il n'y a pas de couches complètement connectées (FC)¹⁵.

FlowNetSimple :

Partie contractive :

Numéro	Type de couche	modification de taille	canaux	filtre
1	POOLING+CONVOLUTION	$(/2 + 10) \times (/2 + 10)$	(x10+4)	7x7
2	POOLING+CONVOLUTION	$(/2) \times (/2)$	x2	5x5
3	POOLING+CONVOLUTION	$(/2) \times (/2)$	x2	5x5
4	CONVOLUTION	/	/	3x3
5	POOLING+CONVOLUTION	$(/2) \times (/2)$	x2	3x3
6	CONVOLUTION	/	/	3x3
7	POOLING+CONVOLUTION	$(/2) \times (/2)$	x2	3x3
8	CONVOLUTION	/	/	3x3
9	POOLING+CONVOLUTION	$(/2) \times (/2)$	x2	3x3

Table 2: Commandes de compression

Partie expansive : 4 couches d'upconvolution.

Ici, la "taille" correspond à la dimension (w x h) d'une feature map. On observe que plus on avance dans les couches, plus la hauteur et la largeur de chaque feature map diminue. En parallèle, on observe que le nombre de canaux augmente à mesure qu'on avance dans la partie contractive. Le "filtre" désigne la taille du masque de convolution utilisé dans chaque couche.

FlowNetCorr :

FlowNetCorr a la même architecture, mais, avec une "couche de corrélation" en plus entre la couche numéro 3 et 4 et les paramètres $k = 0, d = 20, s_1 = 1, s_2 = 2$.

L'algorithme Adam a été utilisé pour l'entraînement. C'est l'algorithme utilisé car le réseau a une convergence plus forte que la descente de gradient stochastique. Ils ont pris comme paramètres : B1=0.9 et B2=0.999 et $\lambda = 10^{-4}$ ("learning rate"), pour des mini-patches de 8 paires d'images. Le

¹⁵En effet, les couches complètement connectées "fixe" le nombre d'entrées et de sorties (avant et après la couche) alors que les couches de convolution font passer un masque de convolution sur une entrée (forcément plus grande que le masque) ce qui permet plus de liberté sur la taille de l'entrée

learning rate est ensuite diminué à partir d'un certain moment pour affiner l'entraînement. Flownet-Corr a en revanche un problème de "gradient explosif". Pour pallier à ça, le learning rate est plus faible au début, puis, augmente jusqu'à 10^{-4} . Une fois une certaine stabilité obtenue, la procédure d'avant (pour FlowNetS) est utilisée. Pour plus de précisions sur les termes utilisés sur l'apprentissage d'un réseau de neurone : voir la section Apprentissage 5.1.4.

Remarque 1 *L'upscaling est une méthode pour augmenter les dimensions d'une paire d'images (ça "augmente" les images) avant traitement par le réseau. L'article indique que cette technique est intéressante pour avoir de meilleurs résultats* ¹⁶.

Fine-tuning (+ft) : Les données MPISintel ont été utilisées pour le fine-tuning avec un learning rate très faible pour un nombre d'itérations optimal obtenu empiriquement (pendant l'entraînement).

5.2.6 Résultats dans l'article

FlowNet est testé sur Sintel, Kitti, Middlebury et Flying chairs. Pour pouvoir comparer les données, ils utilisent le average end-to-end point error (erreur standard pour l'estimation du flux optique) ce qui correspond à la distance euclidienne entre, le flux trouvé et le véritable flux, moyennée sur tous les pixels. Plus le chiffre (EPE) est faible, plus le résultat de FlowNet est proche de la réalité et plus on peut considérer que FlowNet est performant.

En général, on peut observer que FlowNet n'est pas le plus performant sur les données de test par rapport à ses adversaires EpicFlow, DeepFlow,... (voir par exemple les tests sur les dataset SintelClean, Sintel Final ou KITTI). Néanmoins, les concepteurs de FlowNet considèrent que leur réseau est le plus efficace en termes de performances relativement au temps d'exécution. On peut noter que FlowNet est le meilleur réseau sur le dataset Chairs (en considérant les données de test). Ce résultat peut s'expliquer par le fait que FlowNet a été entraîné sur le même dataset (mais sur d'autres données que celles du test). Concernant le dataset Chairs, la version corr de FlowNet (FlowNetC) est un peu plus performante que FlowNetS. Mais, sur d'autres dataset (notamment avec KITTI), la tendance peut s'inverser à cause du fait que les mouvements sur ces datasets peuvent être plus grands/amples. De même, FlowNetS semble être meilleur que FlowNetC lorsqu'il y a des mouvements constants et larges dans l'image comme du brouillard ou des nuages (voir dataset SintelFinal qui en contient). Finalement, FlowNet a une certaine capacité de généralisation entre l'entraînement et de nouvelles données utilisées pour les tests.

Remarque 2 *Expérimentalement, les concepteurs de FlowNet se sont rendus compte de l'importance du nombre de données dans leur dataset : il doit être élevé sinon les résultats ne sont pas satisfaisants.*

Remarque 3 *D'après ses concepteurs, FlowNet semble être performant pour détecter les "détails" sur les images.*

5.2.7 Remarque sur FlowNet2

Le but de l'article de FlowNet2 est d'optimiser au maximum l'EPE de FlowNet1 en explorant différents changements. FlowNetCorr a été réentraîné et donne de meilleurs résultats avec l'entraînement spécifié dans le deuxième article sur FlowNet avec les datasets Flying Chairs et Flying Thing 3D. Aussi, différentes architectures ont été testées où l'architecture "de base" de FlowNet a été dupliqué et "concaténé" avec diverses changements.

¹⁶Dans notre contexte d'optimisation, l'upscaling n'est peut-être pas intéressant car cela demande un pré-traitement et donc plus de calculs

6 Inférence de FlowNet

Nous avons récupéré la première version de FlowNet et nous l'avons fait fonctionner sur l'EM780. Puis nous avons dû tester si FlowNet était vraiment pertinent dans le cadre de ce projet. Le défi avec la capture de météore est le fait que le météore est un objet très petit qui peut facilement ne pas être détecté si on prend en compte la qualité moyenne de certaines vidéos (et donc le bruit dans ces images).

Au début nous avons testé des images sur la version la plus simple de FlowNet, c'est-à-dire FlowNet-Simple avec des poids différents. Nous sommes parvenus assez rapidement à la conclusion que ce réseau de neurone ne donnait pas de résultat assez précis, même si nous avons pu remarquer une très grande amélioration entre de "bons" et "mauvais" poids.

Nous avons continué de tester le flot optique obtenu avec FlowNetCorr, dont les résultats sont fournis ici : 6.3, qui est une version un peu plus compliqué de FlowNet1 mais pas encore aussi complexe que FlowNet2. Les tests réalisés sont faits avec des poids d'une EPE de 1.766 sur les données d'entraînement, ce qui est assez élevé.

6.1 Données testées

Les données que nous avons utilisées ont été fournies par M. Cassagne. Nous avons récupéré des images de vidéo de 30 fps montrant des météores avec un fond de villes ou de nuages, ce qui correspond aux images que pourrait prendre le nanosatellite. Nous avons essayé de modifier les images données au réseau pour voir si cela améliorerait les résultats ou non. Les modifications testées ont été :

- d'images de couleur à en nuance de gris (ce qui est pertinent car les images de la caméra du nanosatellite seront en nuance de gris)
- réduction de qualité (l'espacement des pixels étant diminué, le flot optique aurait pu être plus facilement calculé)
- des images coupées sur le météore (le météore étant très petit sur un fond qui "bouge" dans le même sens, le fait de couper l'image sur le météore permet de donner "plus de poids" au mouvement du météore et le crop fait sens vis à vis du travail fourni par le premier axe de ce projet)

Les météores pouvant apparaître que sur quelques frames, nous n'avons pas cherché à regarder le flot optique sur des images avec un fps moins élevé. Finalement, nous donnons au réseau deux images préalablement extraite d'une vidéo, préselectionnée (ou nous pouvons remarquer un météore à l'oeil nu) et peut être modifiée de, soit 1280x720 (la taille originale), soit 480x320 (crop ou réduite). Pour cela nous avons utilisé des commandes de FFMPEG.

6.2 Sorties du réseau

En sortie du réseau de neurones, nous avons une image de taille divisée par quatre (de la taille initiale) ou la couleur des pixels représente l'angle du vecteur de flot optique calculé et l'intensité des couleurs représente la norme du vecteur. Nous pouvons aussi avoir en sortie des fichiers npy qui donnent des matrices (numpy) avec le flot optique directement. Des options sont possibles dans le code de FlowNet pour n'avoir accès qu'à un type de sortie ou pour avoir les deux en même temps. Cette option n'implique pas un coût de calcul supérieur dans chacun des cas.

Ci-dessous un exemple de sortie sur le flot calculée entre la 148ème et la 149ème frame de la vidéo v99.mp4 fournie :



Figure 34: Image de flot optique



Figure 35: Vecteurs de sortie sur l'image initiale

6.3 Résultats

6.3.1 Réduction d'image

Différence entre un flot optique obtenu à partir de deux images réduites ainsi qu'à leur taille initiale sur la vidéo v99.mp4 :

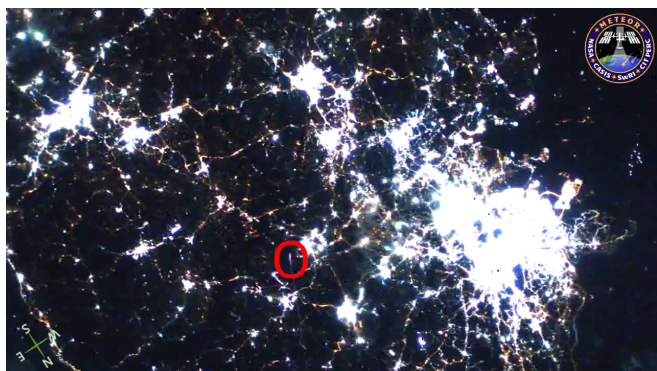


Figure 36: Frame 170

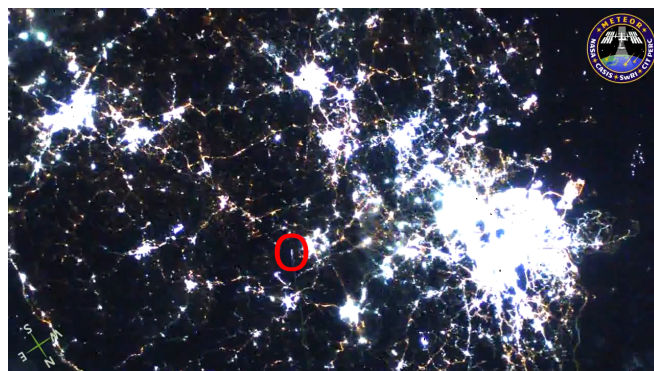


Figure 37: Frame 171

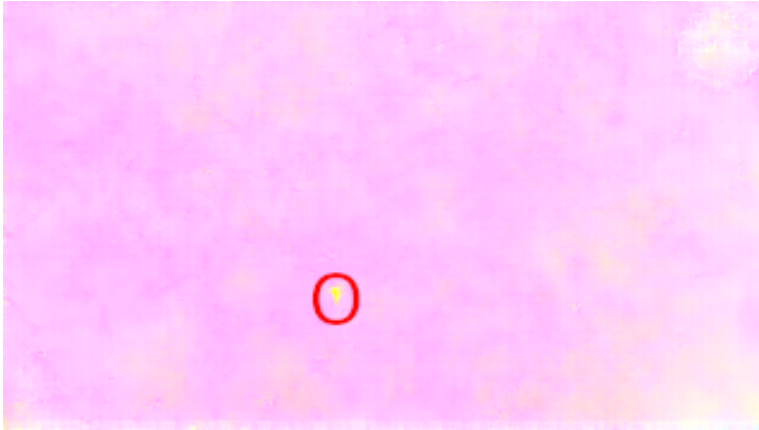


Figure 38: flot optique avec image initiale

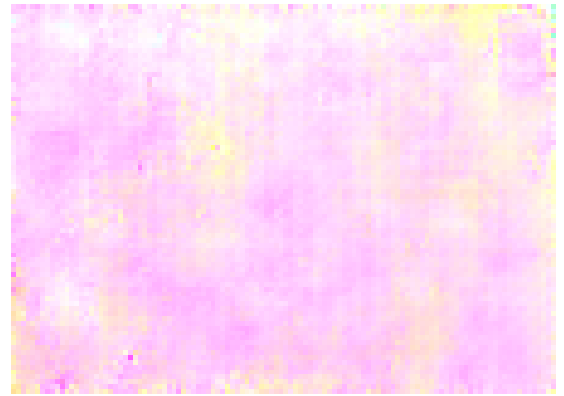


Figure 39: flot optique avec image réduite

Ici nous pouvons remarquer très clairement que le flot optique ne capture pas plus facilement le météore avec une image réduite. La raison serait lié à l'entraînement de FlowNet prenant en compte le bruit gaussien pouvant exister entre deux images : dans une vidéo, des pixels peuvent changer très légèrement de couleur ainsi que disparaître ou apparaître. FlowNet a été entraîné pour ne pas prendre en compte ces "erreurs". Le météore étant très petit, la réduction le rend encore plus petit et donc beaucoup moins détectable.

6.3.2 Coupage et mise en nuance de gris de l'image

Ci-dessous la sortie pour les frames 106 et 107 de la vidéo v03.mp4 avec soit l'image en couleur ou en nuance de gris, soit l'image coupée ou non.



Figure 40: Frame 106



Figure 41: Frame 107

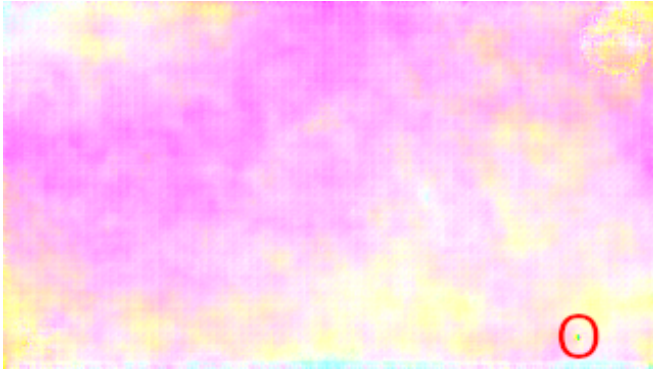


Figure 42: flot optique avec l'image en couleur

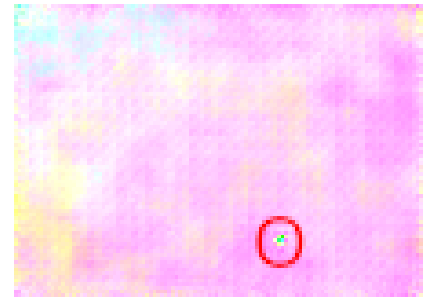


Figure 43: flot optique avec l'image en couleur et coupée

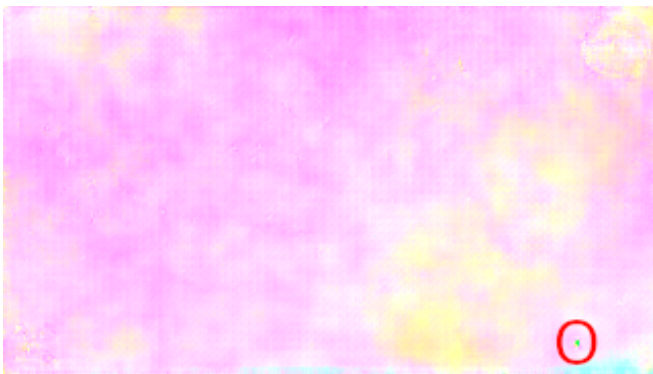


Figure 44: flot optique avec l'image en nuance de gris

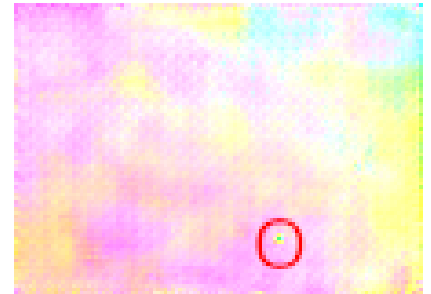


Figure 45: flot optique avec l'image en nuance de gris et coupée

A l'oeil nu, en format vidéo, la différence entre crop/taille initiale et couleur/nuance de gris est visible. Il faut aussi noter que malgré cela quand le météore n'est pas détectée dans la "pire" version possible (image sans changement), il n'est pas détecté dans les autres versions non plus. Vis à vis des performances, le réseau ne gagne pas de temps en traitant une image en nuance de gris (ce qui pourrait être une piste si le but est d'optimiser ce réseau dans ce projet) mais en gagne énormément en coupant l'image. Ainsi, pour la chaine de traitement après la détermination du flot optique et pour le temps de calcul, il serait préférable de couper l'image (la vidéo étant déjà en nuance de gris).

6.3.3 Limites des tests

Ci-dessous le flot calculé à partir de la vidéo v02.mp4, une séquence avec des nuages et un météore très peu visible.



Figure 46: Frame 129



Figure 47: Frame 130

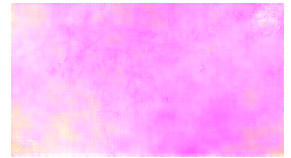


Figure 48: flot optique avec image initiale

Cependant, comme montré au-dessus, dans certaines séquences compliquées, nous n'arrivons pas à détecter de météores. Nous avons remarqué que entre des séquences avec en fond une ville contre des séquences avec en fond des nuages, le réseau pouvait plus facilement trouver le météore avec un fond de type "ville". En regardant plus attentivement, cela fait sens du au fait que les pixels des nuages "bougent" énormément (beaucoup de pixels dans les nuages changent assez brusquement de couleurs alors qu'on peut à l'oeil nu remarquer un mouvement global).

6.3.4 Conclusion

Les images de flot optique obtenues précédemment représentent, pour certaines, un flot optique assez satisfaisant et pertinent par rapport au mouvement étudié. Nous avons testé la pertinence de certaines modifications sur les images en entrée et avons eu des résultats concluant dessus. Néanmoins, pour d'autres images, certains flots optiques demeurent insatisfaisants. On a vu que cette performance pouvait être fortement liée au choix du modèle entraîné mais aussi au choix de différents paramètres sur la paire d'image à l'entrée comme les différents type de fond des vidéos.

Nous avons aussi mesuré le temps nécessaire pour le calcul du flot optique qui est disponible dans un table en annexe. Ce sont des résultats obtenus sur deux vidéos : v00.mp4 et v02.mp4.

Voyant ces résultats, nous avons conclus que les poids que nous avions n'était pas suffisant pour notre projet. Nous avons donc essayé de commencer à générer un dataset pour un apprentissage plus spécifique à notre problème dans le temps qu'il nous restait.

7 Génération d'une base de données

7.1 Motivations et idée générale

Comme expliqué plus haut dans "documentation pour FlowNet", FlowNetSimple a été entraîné à partir d'un dataset de synthèse "FlyingChairs". Ce dataset correspond peu à un environnement spatial utile pour notre mission. Les limites de ce dataset sont les suivantes :

- le fond des images utilisées : paysages terrestre (paysage de montagne par exemple) **versus** paysage spatial
- mouvement des chaises : translations et rotations **versus** mouvement du météore : rectiligne uniquement
- amplitude du mouvement des chaises différente de l'amplitude du mouvement des météores

- taille des objets : une chaise est bien plus grande qu'un météore



Figure 49: Exemple d'une paire d'images issue de "FlyingChairs"

Utiliser un autre dataset plus spécialisé (dans la tâche de la mission) sur lequel entrainer le réseau nous a paru naturel. Ce nouveau dataset spécialisé sera, tout comme "FlyingChairs", synthétique et s'appuiera sur une petite base de données de 140 vidéos de météores capturés depuis l'espace.

Idée générale : Dans un premier temps, on extrait à partir de la base de données de 140 vidéos 2 types de choses : les paysages spatiaux (qui joueront le rôle de "background" par la suite) et les météores (qui joueront le rôle de motifs à coller par la suite). Une fois que nous avons constitué 2 banques de données avec d'un côté les backgrounds et de l'autre les motifs, on génère des paires d'images aléatoires en collant un motif sur un background. Enfin, on génère le fichier ".flo" associé à chaque paire qui correspond au flot optique entre les 2 images de la paire. Ce dernier fichier jouera le rôle de "Ground Truth" (GT) dans l'apprentissage.

Remarque : Pour constituer la banque des motifs, on cherche le moment dans chaque vidéo où le météore est le plus visible, on fait un arrêt sur image et on détourne le météore.

7.2 Génération d'une paire d'images

Pour que le dataset soit le plus réaliste possible on a introduit 2 types de mouvements dans les paires d'images : le mouvement de translation du background (qui traduit le déplacement de la caméra) et le mouvement rectiligne du motif plus ample que la translation du background (correspond au déplacement du météore).

Pour commencer la génération, on choisit aléatoirement : le background parmi une banque de 275 images et le motif parmi une banque de 50 images. L'objectif est donc de créer une paire de 2 images : imageA et imageB à partir du background et du motif choisis.



Figure 50: Exemple de background



Figure 51: Exemple de motif

7.2.1 Translation du background

Afin d'introduire un mouvement de translation pour le fond, le background est de base plus grand que la taille finale de la paire d'images. Cela nous permet d'effectuer une translation.

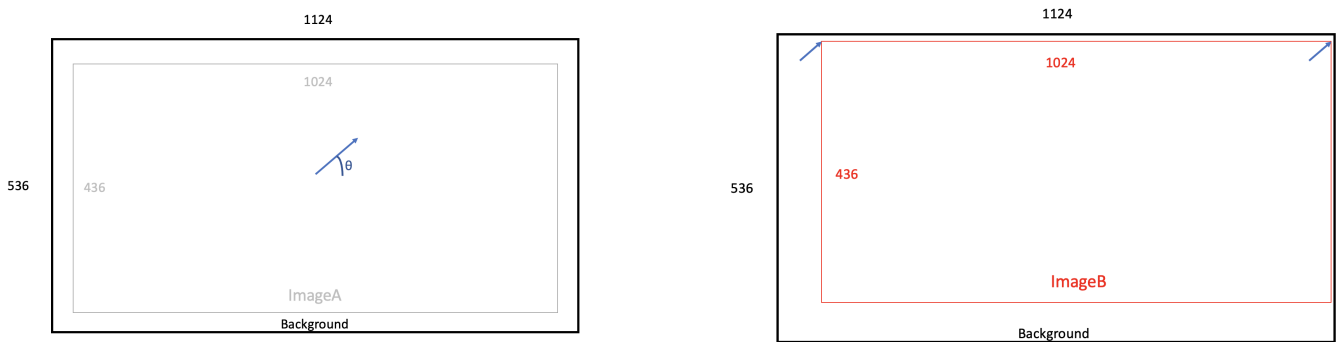


Figure 52: Illustration de la translation du background

L'imageA se situe toujours au centre du background. L'imageB est l'image recadrée (de même taille que l'imageA) suivant la translation induite par l'angle θ

Ici l'angle θ est choisi aléatoirement tout comme l'amplitude de déplacement entre l'imageA et l'imageB (c'est-à-dire la norme du vecteur de déplacement). En pratique, on fait varier l'amplitude de déplacement entre 0 et 2 pixels. Dans 40% des cas l'amplitude vaut 0, pour le reste des cas l'amplitude vaut 2 pixels.

7.2.2 Mouvement du motif

Après avoir généré un déplacement pour le background, il est nécessaire de coller le motif sur l'imageA et de lui créer un déplacement pour ensuite répercuter celui-ci sur l'imageB. Pour générer des mouvements de météores diversifiés, on procède de la manière suivante :

- on choisit une position initiale aléatoire du météore sur l'imageA (suivant une loi uniforme sur la largeur et la hauteur)
- on choisit une direction de déplacement du météore aléatoire suivant l'angle α (loi uniforme)
- on choisit une amplitude de déplacement aléatoire du météore (loi uniforme) entre 4 et 10 pixels.

Remarque : on introduit un padding pour la position initiale afin d'éviter que le météore soit sur le bord de l'image.

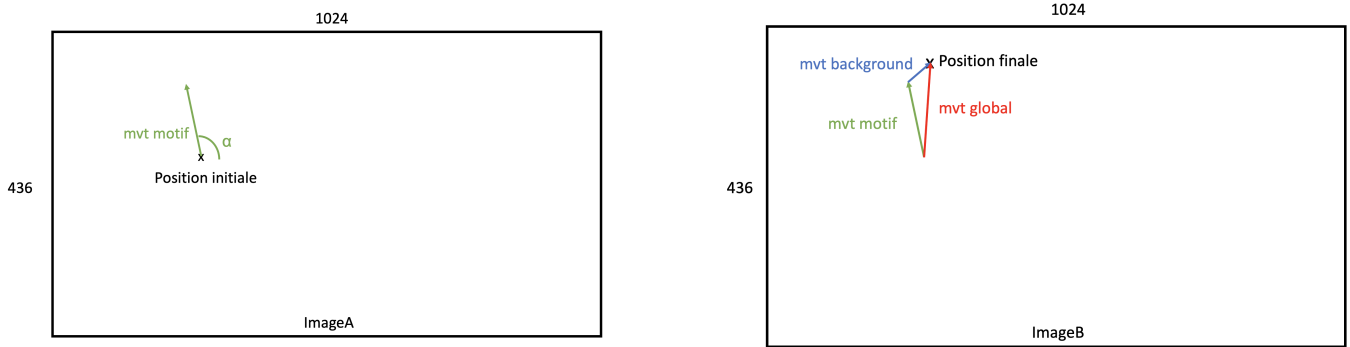


Figure 53: Illustration du mouvement du motif

Finalement, pour obtenir l'imageA on colle le motif à la position initiale. Pour obtenir l'imageB, on colle le motif à la position initiale + le déplacement du motif en tenant compte du déplacement du background (déterminé précédemment)



Figure 54: Exemple d'une paire obtenue à partir du background et du motif sélectionnés

7.2.3 Génération du flot optique associé

Après avoir entièrement déterminé le mouvement du background ainsi que celui du motif, on peut facilement calculer le flot optique associé à la paire d'images car on connaît le déplacement de chaque pixel.

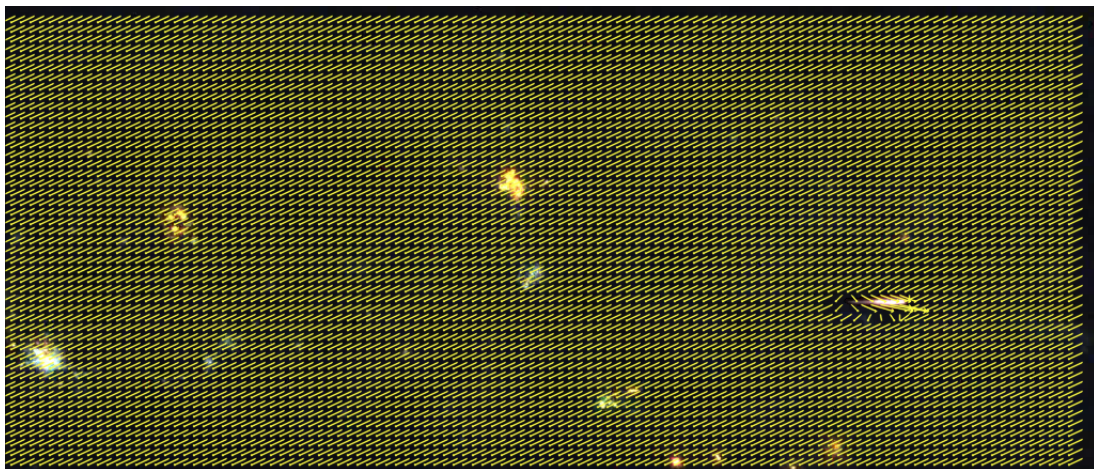


Figure 55: Flot optique obtenu à partir de la paire exemple

Dans cet exemple, θ vaut environ 270° tandis que α vaut environ 0° .

8 Conclusion

Les objectifs du projet Meteorix permettent la détection efficace du mouvement dans les vidéos et l'amélioration de la qualité d'image à l'aide du réseau de neurone FlowNet. Grâce à l'intégration de ces technologies, nous avons pu isoler les parties pertinentes des séquences vidéo tout en améliorant visuellement les images.

8.1 Impacts éthiques/sociétaux

Préoccupations :

- Détournement à des fins militaires : Il existe un risque d'utiliser la détection de météores dans des contextes de défense ou de guerre, ce qui pourrait mener à des applications militaires non souhaitées.
- Manipulation des résultats : Il y a une possibilité de fausser les résultats concernant la caractérisation des météores, ce qui pourrait compromettre leur étude et leur gestion.
- Concurrence entre pays : La compétition entre nations pour envoyer le plus grand nombre de nanosatellites pourrait mener à une nouvelle forme de guerre froide, générant ainsi une pollution accrue dans l'espace.

Bienfaits :

- Réduction des déchets spatiaux : Une meilleure compréhension des déchets spatiaux pourrait sensibiliser le public et les autorités, permettant ainsi de mieux les gérer et de réduire leur accumulation dans l'espace.
- Optimisation de l'environnement spatial : Remplacer les satellites traditionnels par des nanosatellites offrirait un environnement plus propre et permettrait d'optimiser l'utilisation de l'énergie, ce qui pourrait améliorer les résultats scientifiques.
- Réduction de l'impact écologique : Utiliser des matériaux en quantités limitées pour construire des nanosatellites permettrait de réduire la pollution liée à l'extraction des ressources naturelles nécessaires à leur fabrication.
- Avancées technologiques : La contrainte énergétique dans la conception des nanosatellites pousse à des innovations technologiques majeures, permettant d'effectuer une multitude d'opérations dans l'espace tout en utilisant peu de ressources.

8.2 Organisation

Notre équipe se retrouve hebdomadairement au créneau réservé au projet industriel sur l'emploi du temps pour travailler sur le projet ou sur les parties management liées au projet. Nous utilisons WhatsApp pour la communication entre nous ainsi qu'avec la tutrice académique. Nous utilisons Mattermost sinon, lorsqu'il s'agit de la communication avec le client. Nous programmons aussi des rendez-vous avec lui régulièrement pour lui donner des nouvelles sur l'avancement du projet. Nous avons aussi mis en place un github pour le partage des codes et un autre github pour le transformer en page de documentation du projet.

Nous nous sommes décomposés en deux groupes avec des missions spécifiques. Le groupe MVE (Cyprien, Anes, Loïc) travaille sur l'algorithme de détection de météores, FFMPEG et l'optimisation

énergétique de cet algorithme, tandis que le groupe Flownet (Alex et Paul) travaille sur le réseau de neurones FlowNet et son entraînement avec une base de donnée choisie.

Dans le groupe MVE, chacun a travaillé sur motion vector extractor et l'étude de ce code qui permet la détection de vecteurs vitesses à partir d'une vidéo donnée qui est écrit en langage Python et C++. Ensuite il y a eu une répartition de tâches spécifiques, Anes a réalisé une version de motion vector extractor entièrement en C++, Cyprien et Loïc ont travaillé sur l'em780 avec l'utilisation de FFMPEG en ciblant des GPU spécifiques. Cyprien a ensuite réalisé l'évaluation des performances avec les différents algorithmes motion vector extractor ainsi que des notebooks sur l'utilisation de mve et de FFMPEG et Loïc s'est occupé du ciblage sur VAAPI et a effectué un mkdocs pour documenter le projet.

Dans le groupe Flownet, n'étant que deux, il était plus facile de se répartir les tâches en fonction des affects. Nous avons commencé par se documenter sur les CNNs et le flot optique et avons fini de rédiger la documentation disponible dans le rapport (5). Paul s'est concentré sur faire marcher FlowNet. Il a rédigé la documentation quant à l'installation de Flownet sur une machine du Lip6 et a lancé des tests sur FlowNetS. Il a commencé à créer un dataset sur des météores. Alex a lu et commencé la documentation sur l'article de FlowNet. Il a lancé les tests sur FlowNetCorr.

8.3 Pistes de recherche pour la suite

Du côté de **Motion Vector Extractor** nous avons plusieurs idées à tester. Dans un premier temps il serait bien de réessayer à adresser le GPU Radeon qui est quasi-identique à la puce envoyé dans l'espace. Cela nous permettra de tester notre algorithme en "conditions réelles".

Ensuite, nous pourrions améliorer notre API en développant une fonction qui vérifie sur plusieurs images successives si des vecteurs ont la même direction et se déplacent "légèrement", tel un météore. Cela permettra d'affiner les zones d'intérêt et pourrait aussi supprimer les éclairs.

Enfin, il serait intéressant de lancer notre algorithme sur un plus grand nombre de vidéos qui ont été compressées en utilisant les paramètres de compression. Cela nous permettrait d'avoir plus de données pour tirer plus de conclusions à partir de l'ACP. Nous pourrions de plus imaginer faire de la classification supervisée en utilisant une RandomForest par exemple.

Pour ce qui est de l'axe "**FlowNet**", nous avons essayé de tester FlowNet2 après avoir déterminé que FlowNetCorr ne marchait pas assez bien pour les séquences compliquées. Par manque de temps, nous n'avons pas réussi à l'implémenter correctement ce qui pourrait être une piste intéressante surtout que FlowNet2 offre une très large gamme de CNN avec des buts un peu différents (des optimisations différentes).

Aussi, il serait pertinent d'entraîner FlowNetCorr avec les learning rate donnée dans l'article FlowNet2 qui montre une très grande marge de progression (et de potentiellement l'entraîner sur Flying Chairs et Flying Things 3D).

Ce serait intéressant de continuer à construire et perfectionner le dataset sur les météores en cherchant plus de motifs. De même, il faudrait augmenter la qualité du détourage (depuis les vidéos réelles) des motifs (météores) afin de les rendre plus réaliste. On pourrait aussi imaginer qu'au lieu de prendre un seul motif et juste de le traduire entre l'imageA et l'imageB (cf partie génération d'une base de données), d'avoir une paire de motifs et de coller le motifA sur l'imageA et le motifB

sur l'imageB. Par ailleurs, une piste intéressante serait de créer synthétiquement les motifs en jouant sur l'opacité de la couleur blanche. Aussi, dans l'article sur FlowNet1, les créateurs avaient parlé de leur génération du dataset Flying Chairs où ils modifient toujours légèrement les images pour éviter l'overfitting (en introduisant du bruit gaussien), ce qu'il faudrait prendre en compte dans le dataset sur les météores.

Pour ce qui est de la documentation, l'article de FlowNet date d'il y a 10 ans, il serait pertinent (au moins pour la génération de dataset) de regarder des articles plus récents. Dans l'article de FlowNet2, un article sur Flying Things 3D a été mentionné. Lire l'article de FlowNet2 semble aussi être une bonne idée.

References

[◇] Sources principales :

Sorbonne Université : [Projet nanosatellite Meteorix](#),
Code source de [Motion vector extractor](#),
Code source de [FlowNet](#)

[◇] Sources complémentaires :

Article sur la première version de [FlowNet](#)
[Cours](#) sur la mesure du **flot optique**, chapitre 3 section définition du flot,
Site sur les [CNN](#)
Site de vulgarisation sur le [unpooling](#)
Site sur le [traitement d'images](#)
Article sur la méthode [upconvolution](#)
Article sur l'algorithme [coarse to fine scheme](#)
Site de vulgarisation sur [l'algorithme d'Adam](#)
Article sur [l'algorithme d'Adam](#)
Deep learning and practice with Mindspore de Lei Chen

taille des img	nb d'img	cpu/gpu	Résultat	temps mis	nb de frame par sec calculée
1280x720	9	cpu	non concluant	81,99	0,110
1280x720	9	cpu	non concluant	81,70	0,110
1280x720	16	cpu	passable	143,23	0,112
1280x720	16	cpu	passable	143,28	0,112
1280x720	26	cpu	concluant	230,23	0,113
1280x720	26	cpu	concluant	230,11	0,113
1280x720	26	cpu	concluant	230,98	0,113
1280x720	53	cpu	concluant	465,88	0,114
1280x720	53	cpu	concluant	467,30	0,113
1280x720	53	cpu	concluant	467,24	0,113
1280x720	9	gpu	non concluant	5,81	1,549
1280x720	9	gpu	non concluant	5,50	1,638
1280x720	16	gpu	passable	7,29	2,195
1280x720	16	gpu	passable	6,63	2,413
1280x720	26	gpu	concluant	6,93	3,755
1280x720	26	gpu	concluant	6,78	3,837
1280x720	26	gpu	concluant	7,53	3,451
1280x720	53	gpu	concluant	11,64	4,552
1280x720	53	gpu	concluant	13,02	4,072
1280x720	53	gpu	concluant	11,26	4,708
448x320	9	cpu	non concluant	13,72	0,656
448x320	9	cpu	non concluant	13,72	0,656
448x320	16	cpu	passable	22,26	0,719
448x320	16	cpu	passable	22,03	0,726
448x320	26	cpu	concluant	31,66	0,821
448x320	26	cpu	concluant	33,98	0,765
448x320	26	cpu	concluant	34,19	0,761
448x320	53	cpu	concluant	67,76	0,782
448x320	53	cpu	concluant	66,43	0,798
448x320	53	cpu	concluant	66,58	0,796
448x320	9	gpu	non concluant	4,29	2,096
448x320	9	gpu	non concluant	4,28	2,102
448x320	16	gpu	passable	4,59	3,490
448x320	16	gpu	passable	4,50	3,553
448x320	26	gpu	concluant	4,71	5,523
448x320	26	gpu	concluant	4,66	5,578
448x320	26	gpu	concluant	4,93	5,276
448x320	53	gpu	concluant	5,62	9,431
448x320	53	gpu	concluant	5,85	9,064
448x320	53	gpu	concluant	7,22	7,341

Table 3: Résultat de FlowNetCorr